Episodio 13 de 14: Juicio a la réplica (2): corrección, RAM, estadística y sentencia

Serie «Leer sin que te quiten el libro»: la réplica del paper «A Fast Wait-Free Solution to Read-Reclaim Races in Reference Counting» (Euro-Par 2024), explicada para principiantes.

- **Episodio anterior:** episodio 12, «Juicio a la réplica (1): escala, método y Folly».
- **Episodio siguiente:** episodio 14, «Ensayo para la exposición: las preguntas del jurado».

Lo esencial de este episodio:

- Cargos 8 a 14: la corrección, la RAM, la selección de resultados, la estadística, la máquina sin controlar, el uso de IA y las erratas del informe.
- La sentencia: es una reproducción válida y honesta a escala de un PC; confirma la idea central y discute con datos lo que no reproduce.
- Mejoras por orden: corregir las erratas antes de exponer; luego un segundo módulo de RAM, intervalos de confianza y la figura 5 con la receta exacta; y más lejos, una máquina de varios sockets.

Cargo 8. «“0 errores” no demuestra que sea correcto»

Juez escéptico: Decís que el algoritmo es correcto porque ASan y TSan no encontraron nada. Una prueba solo ve lo que pasa mientras mira.

Juez defensor: Totalmente de acuerdo, y el documento lo dice con esas palabras: «encontrar 0 errores no demuestra que no existan». Lo que sí demuestran las pruebas es esto (Parte 6.5):

- **El detector funcionaba.** La versión ingenua lo hace saltar enseguida con un `double-free`. Y los binarios llevaban la instrumentación: se comprobó.
- **Las reglas del paper no son adornos.** Si se rompe a propósito la regla C ≥ 2P, aparecen 42 348 errores.
- **La cota *wait-free* se cumple con margen.** Forzando el caso extremo (P = 32), en 8 061 142 recargas el máximo fue de **8 intentos**, cuando la cota es 32.
- **Los rivales tampoco rompen la memoria:** 26 pruebas, 0 avisos.

La demostración de que el algoritmo es correcto está en el paper. Lo que nosotros comprobamos es que **su implementación** se comporta como dice.

Juez escéptico: Son 4 segundos por prueba y un solo escenario. Y Folly y Anderson no pasaron TSan.

Juez defensor: Correcto: `libfolly.a` no está instrumentada y daría avisos falsos. Está en la Parte 8.

Presidencia: concedido, pero la réplica ya lo concede. El veredicto dice «Se reproduce **en lo probado**». Esa frase es exactamente la correcta.

Cómo blindarlo: - Pruebas de estrés más largas, de minutos u horas, y en más escenarios. - TSan en Folly y Anderson, compilando Folly con `-fsanitize=thread`. Lleva tiempo de

compilación. - Si queréis subir de nivel, **comprobación de modelos** (por ejemplo, con GenMC) del núcleo del algoritmo con P y C pequeños. Explora todos los órdenes posibles entre hilos, no solo los que salen por azar.

En 20 segundos: «Tienes razón, una prueba no demuestra que sea correcto: eso lo hace la demostración del paper. Nosotros comprobamos que el detector funcionaba, que romper sus reglas produce errores y que la cota se cumple: 8 intentos como mucho frente a 32.»

Cargo 9. «Con un millón de pilas solo medisteis vuestra RAM»

Juez escéptico: Con n = 10^6, Autores, Anderson, Libstdc++ y Folly empatan entre 53 y 57 Mop/s, y el paper da a Autores como ganador. Vuestro PC tiene un solo módulo de RAM. La fila 5, «No se reproduce», es culpa de vuestra máquina.

Juez defensor: Probablemente sí, y por eso la fila 5 dice «No se reproduce **en este PC**». Pero no lo suponemos: **lo medimos** con los contadores de la controladora de memoria (Parte 6.1):

- El techo de este PC, leyendo líneas al azar, es de unos 37 GB/s.
- Con 24 hilos, Anderson llega al 93 % de ese techo, Libstdc++ al 92 % y Folly al 89 %.
- Al alinear las pilas, cada operación saca de la RAM entre 207 y 265 bytes más, y el rendimiento cae casi en la misma proporción (Parte 6.6). Cuando la RAM está al límite, más bytes por operación significa menos operaciones.

Juez escéptico: Pero Autores rinde lo mismo usando solo el 75 % del techo. Eso no lo explica la RAM.

Juez defensor: Correcto, y el documento lo dice: «no lo explica todo». Algo más frena a Autores; quizá el tiempo de espera de cada acceso, que no se midió. Por eso la conclusión es que la medición «refuerza» la explicación de la RAM, **no que la demuestre**.

Presidencia: concedido en parte. Es probable que la fila 5 dependa de la máquina, y el documento lo reconoce con cuidado. Falta la prueba definitiva.

Cómo blindarlo: poner un segundo módulo de RAM igual (DDR5-5600) y repetir `herramientas/medir_memoria.py` y el panel n = 10^6. Si el rendimiento sube, queda demostrado. Es probablemente **la mejora con mejor relación entre coste y valor** de esta lista.

En 20 segundos: «Es muy probable, y por eso decimos “no se reproduce en este PC”. Lo medimos: tres algoritmos llegan al 89-93 % del techo de la RAM. La prueba definitiva sería un segundo módulo de memoria.»

Cargo 10. «Elegisteis lo que os convenía»

Juez escéptico: Es fácil reproducir un paper si solo enseñas lo que sale bien.

Juez defensor: Hicimos lo contrario:

- Los **6 escenarios** de la figura 5 y las **3 operaciones** de la figura 6, todos.
- Las **15 afirmaciones** del paper, una por una, incluidas las que no salen: 3 «No se reproduce» y 1 «No se confirma».

- Activamos a **Williams en la latencia**, que el paper omitió diciendo que «casi siempre era mayor». En vez de creer la frase, la comprobamos: sale «en parte».
- Los **datos descartados se conservan**, con la explicación de por qué se descartaron.
- Cada punto se midió en **orden aleatorio**, en un proceso nuevo.

Presidencia: a favor de la réplica. Es difícil acusar de selección a un trabajo que publica sus resultados negativos y sus propios errores.

Cómo blindarlo: en la exposición, dedica **una diapositiva a lo que no se reproduce**. Los jurados lo valoran mucho.

En 20 segundos: «Probamos las 15 afirmaciones, y 4 no salen; las contamos igual. Incluso medimos a Williams en latencia, que el paper había omitido.»

Cargo 11. «Sin intervalos de confianza no hay ciencia»

Juez escéptico: Dais medianas y rangos de la peor a la mejor ronda. No hay intervalos de confianza ni pruebas estadísticas. ¿Cómo sabéis que una diferencia es real?

Juez defensor: Usamos una regla **conservadora**: si los rangos de las rondas se solapan, es empate. Y solo damos por buena una ventaja grande, de 2 o 3 veces, porque el recorte puede mover un punto hasta un 19 % (cargo 3). Para el orden general usamos la τ de Kendall, que siempre sale positiva: de +0,20 a +0,80 (Parte 6.1).

Juez escéptico: Conservadora, sí. Formal, no.

Presidencia: concedido en parte. La regla del solapamiento es prudente y está bien aplicada, pero un jurado de estadística pedirá algo más formal. Es fácil de añadir.

Cómo blindarlo: añadir a `analizar.py` **intervalos de confianza por *bootstrap*** para las medianas y los cocientes. Consiste en remuestrear las rondas miles de veces y ver cuánto se mueve la mediana. Son unas 30 líneas de Python. Con 5 rondas los intervalos saldrán anchos, y eso también es información honesta.

En 20 segundos: «Usamos una regla conservadora: si los rangos se solapan, es empate, y solo contamos ventajas de 2 o 3 veces. Añadir intervalos por bootstrap es una mejora que tenemos identificada.»

Cargo 12. «La máquina no estaba controlada»

Juez escéptico: Es un PC de escritorio. No fijasteis la frecuencia, el turbo cambia solo, la CPU es híbrida y llegó a 99 °C.

Juez defensor: Todo eso es cierto y está en la Parte 8:

- **Temperatura:** solo 5 de las 3 720 ejecuciones con Docker y 26 de las 9 796 sin Docker llegaron a 90 °C o más. Están repartidas y se conservan.
- **Orden aleatorio:** cualquier interferencia afecta a todos los algoritmos por igual, en promedio.
- **CPU híbrida:** sin fijar los hilos, Linux los reparte entre núcleos P y E. Por eso la batería de afinidad sí los fija.

Presidencia: concedido en parte. El orden aleatorio reparte el efecto, pero no lo elimina. Las conclusiones grandes no dependen de esto; las diferencias pequeñas, quizá sí, y ya se tratan como empate.

Cómo blindarlo: gobernador `performance`, turbo desactivado y núcleos aislados del sistema. Luego, repetir un subconjunto alternando con la configuración normal, para ver si algo cambia.

En 20 segundos: «No fijamos la frecuencia, es cierto. Lo compensamos con el orden aleatorio y registramos la temperatura de cada ejecución: menos del 0,3 % llegó a 90 °C.»

Cargo 13. «¿Esto lo hicisteis vosotros o una IA?»

Juez escéptico: El repositorio tiene decenas de guiones y un documento de 1 500 líneas, y en la Parte 8 se menciona «la sesión del asistente». ¿Lo entendéis, o lo ha hecho una máquina?

Juez defensor: Lo que da validez a una réplica no es quién tecleó cada línea, sino dos cosas:

1. **Que cualquiera pueda repetirla.** Cada experimento se lanza con una orden, que el README describe con su tiempo y su estado. Cada cifra sale de un CSV. Hay una comprobación automática sin IA, `herramientas/comprobar_final.py`.
2. **Que el equipo pueda explicarla.** Eso no lo da ninguna herramienta.

Juez escéptico: Lo primero lo cumplís. Lo segundo lo veremos con las preguntas.

Presidencia: depende de vosotros. Si el equipo puede explicar cada parte y ha declarado la ayuda según las normas del curso, no es un problema. Si no puede, este es **el cargo más peligroso de todos**, porque ningún dato lo defiende.

Cómo blindarlo: - **Declarar el uso de herramientas** como pidan las normas del curso. Esconderlo es mucho peor que decirlo. - Que **cada miembro repita una parte** con sus manos y sepa explicarla sin mirar. Por ejemplo, David la cota de `CAS`, Jheremy la figura 5 y Diego la memoria retenida. - Ensayar la sección 6 en voz alta, con alguien que haga de jurado.

En 20 segundos: «Usamos herramientas, y lo declaramos. Pero todo se puede repetir con una orden y cada cifra sale de un CSV. Pregúntanos lo que quieras de cualquier parte.» Y después, que se note que lo sabéis.

Cargo 14. «Vuestro informe contradice vuestros datos»

Juez escéptico: He leído vuestro informe. En `replica/informe/02_latencia.md` decís que el `load` de Libstdc++ con p = 56 tarda 86 700 ns, y la figura 6 del paper dice 99,7 µs. El README da 34,3 Mop/s en un sitio donde el CSV dice 39,126. Si no cuadran vuestros propios números, ¿por qué iba a creer el resto?

Juez defensor: Esos errores están localizados: la Parte 8 de EXPLICACION.md tiene una tabla, «Cosas que el equipo debería revisar». No se corrigieron porque esos archivos tienen responsables en el equipo.

Juez escéptico: Haberlos localizado no los arregla. El día de la exposición, el informe dirá lo que dice.

Presidencia: en contra, si no se corrige antes. Es el cargo más fácil de evitar y uno de los que más daño hace, porque un jurado que encuentra una errata empieza a desconfiar de todo. **Arreglarlo es cosa de horas.**

Cómo blindarlo: que cada responsable corrija su parte usando la tabla de la Parte 8:

Dónde	Qué corregir
`replica/informe/02_latencia.md`, línea 18	86 700 ns → 99,7 µs, como en la figura 6 del paper
`README.md` de la raíz	Los números que no coinciden con `capturas/rendimiento.csv`
`codigo/ejecutar_pruebas.sh`	Añadir `setarch -R` al paso de TSan, como ya hace `correr_tsan.sh`
`codigo/bench.cpp`, línea 73	Decidir si se alinea `Pila` con `alignas(128)`
Parte 7, fila 11	Decidir si se queda en «No se confirma» o pasa a «Dudoso»

En 20 segundos: solo vale una respuesta: «Ya está corregido».

---

4. La sentencia

Presidencia: Tras escuchar los 14 cargos, el tribunal resuelve así:

Criterio	Valoración	Por qué
Artefactos disponibles	**Sí**	Todo está en el repositorio, con los CSV, las figuras y los guiones
Funcionan	**Sí**	Cada experimento es una orden, se reanuda solo, y la versión sin Docker se probó de principio a fin sin fallos
Reutilizables	**En parte**	Las listas de núcleos están escritas para el Core Ultra 9 285; en otra CPU hay que medir primero el mapa de latencias
Resultados reproducidos	**En parte, y bien argumentado**	9 de 15 afirmaciones, más 2 en parte. Lo central del paper (correcto, *wait-free*, sin basura, lectura rápida) se sostiene
Transparencia	**Alta**	Errores propios documentados, datos descartados conservados, hipótesis marcadas como tales
Alcance	**Limitado, y declarado**	Nada sobre 4 sockets ni NUMA; la latencia con contención es débil; la RAM es de un solo módulo

Fallo final: es una **reproducción válida y honesta a escala de un PC**. Confirma la idea central del paper y discute con datos lo que no reproduce. No sirve para hablar de 4 sockets

ni de NUMA, y ella misma lo dice.

Sus dos puntos más fuertes: el argumento de Folly con un solo hilo (cargo 4) y haber medido en vez de suponer: Docker, los recortes, la RAM y la distancia (cargos 2, 3, 6 y 9).

Sus dos puntos más débiles: la latencia con contención (cargo 5) y, si no se corrigen antes, las erratas del informe del equipo (cargo 14).

---

5. Plan de mejoras, por orden

Ordenado por lo que más rinde para el tiempo que cuesta.

Antes de la exposición (horas)

Mejora	Cargo que debilita	Cómo
Corregir las erratas del informe y del README	14	Tabla del cargo 14
Decidir la fila 11 y el `alignas(128)` de `bench.cpp`	7 y 14	Decisión del equipo
Declarar el uso de herramientas según las normas del curso	13	Una frase en el informe y en la exposición
Una diapositiva de «lo que no se reproduce y por qué»	4 y 10	Fila 3 y Folly con un hilo (Parte 6.1)
Diapositivas de reserva para las preguntas difíciles	Todos	Figuras: `fig5_paper_frente_a_replica.png`, `afinidad_dosis_respuesta.png`, `docker_frente_a_nativo_oficial.png`, `memoria_ram.png` y `nativo_alineado_fig5.png`
Ensayar la sección 6 en voz alta	13	Uno pregunta, otro responde, y se cambian los papeles

Si hay una semana

Mejora	Cargo que debilita	Esfuerzo	Cómo empezar
Segundo módulo de RAM	9	Bajo, si alguien presta uno	Repetir `herramientas/medir_memoria.py` y el panel n = 10^6
Intervalos de confianza por *bootstrap*	11, 3 y 5	Bajo: una tarde	Una función en `analizar.py` que remuestree las rondas
Figura 5 completa con la receta exacta	3	Una noche, unas 5 h	Ampliar el experimento `paridad` de `baterias.py` a todos los hilos
Latencia con frecuencia fija y núcleos aislados	5 y 12	Medio: hay que reiniciar con otros	Gobernador `performance`, turbo fuera, `isolcpus`, y

Mejora	Cargo que debilita	Esfuerzo	Cómo empezar
		parámetros del núcleo	repetir la figura 6 con 15 rondas
Otro miembro repite una parte en su PC	7 y 13	Medio	`correr_docker.sh` en otra máquina, adaptando las listas de núcleos
Williams con Boost 1.74 fuera de Docker	Pregunta abierta	Bajo o medio	Compilar `bench_stack` fuera con Boost 1.74, o dentro con 1.83
Escribir a los autores sobre Folly	4	Muy bajo: un correo	Preguntar versión y configuración de Folly en la figura 5

Si queréis llevarlo más lejos

Mejora	Cargo que debilita	Esfuerzo
Máquina de 2 o 4 sockets (servidor de la universidad o nube)	1 y 2	Alto: acceso o dinero
Batería de afinidad en un AMD Ryzen con dos chiplets	1 y 2	Medio, si hay acceso a uno
TSan en Folly y Anderson	8	Alto: hay que compilar Folly instrumentado
Pruebas de estrés largas y en más escenarios	8	Bajo de hacer, largo de esperar
Comprobación de modelos (por ejemplo, GenMC) con P y C pequeños	8	Alto: hay que aprender la herramienta
Contadores de hardware (`perf`) para Folly y para Autores con pocos hilos	4 y 9	Medio: instalar `perf` para este núcleo